

Final Project:

Predicting Building Energy Consumption (Site EUI)

WiDS Datathon 2022

- A. Data Preprocessing
- B. Model Comparison - Baseline and Ensemble Methods
- C. Advanced Models - XGBoost and LightGBM
- D. Hyperparameter Tuning
- E. Visualization and Feature Analysis
- F. Conclusion

A. Data Preprocessing

1. Introduction to the dataset

For this project, I use the WiDS Datathon 2022 dataset. This dataset features building energy usage across multiple U.S. states, including approximately 75,757 building-year observations in the training set with 64 variables:

- **id**: a unique identifier for each building observation
- **State_Factor**: anonymized U.S. state where the building is located
- **building_class**: classification of the building (e.g., residential, commercial)
- **facility_type**: the specific facility type of the building
- **floor_area**: total floor area of the building (sq ft)
- **year_built**: the year the building was constructed
- **energy_star_rating**: ENERGY STAR rating of the building
- **elevation**: elevation of the building location
- **heating_degree_days** / **cooling_degree_days**: climate indicators for heating and cooling demand
- **avg_temp**, **max_temp**, **min_temp**: temperature statistics at the building's location
- **days_above_*** / **days_below_***: counts of extreme temperature days
- **snowfall_inches**, **snowdepth_inches**: snow-related climate features
- **site_eui**: **target variable** - Site Energy Usage Intensity (kBtu per sq ft per year)

Two datasets are provided:

- **train.csv**: 75,757 rows $\times$ 64 columns (includes **id** and target **site_eui**)
- **test.csv**: 9,705 rows $\times$ 63 columns (includes **id**, no **site_eui**)

2. Missing values

Missing values are present across several numeric and categorical columns. To handle these:

- **Numeric columns**: Missing values were imputed using the **median** strategy. Median imputation is preferred over mean imputation because it is robust to outliers, which are common in energy usage data.
- **Categorical columns**: Missing values were imputed using the **most frequent** (mode) strategy.

3. Variable types and encoding

Out of 64 features, only 3 are categorical: `State_Factor`, `building_class`, and `facility_type`. The remaining 61 features are numerical.

Categorical variables were converted to numeric using **Label Encoding**, which assigns an integer to each unique category. While Label Encoding introduces an ordinal relationship, it works well with tree-based models, which are the primary models in this project.

All numeric features were standardized using **StandardScaler**, which centers each feature to mean 0 and standard deviation 1. This is essential for models sensitive to feature scale, such as Ridge Regression and SVR.

4. Train/validation split

The preprocessed training data was split into:

- **Training set:** 80% of samples (~60,606 rows) - used to fit the models
- **Validation set:** 20% of samples (~15,151 rows) - used to evaluate model performance

The split was performed with `random_state=42` for reproducibility.

B. Model Comparison - Baseline and Ensemble Methods

1. Goal

The goal is to predict **Site Energy Usage Intensity (site_eui)** - a normalized measure of a building's annual energy consumption per square foot. The evaluation metric is **Root Mean Squared Error (RMSE)**.

The `id` column was removed as it is irrelevant to prediction. All remaining features were used as predictors.

2. Baseline models

Two linear models serve as baselines:

Linear Regression - A simple ordinary least squares model with all predictors. This model assumes a linear relationship between features and the target. It provides a benchmark to measure how much improvement more complex models can achieve.

Ridge Regression ($\alpha = 1.0$) - Adds L2 regularization to Linear Regression. This penalizes large coefficients, which helps when features are correlated (multicollinearity). Given that many weather-related features in the dataset are correlated (e.g., heating degree days and average temperature), Ridge is expected to improve over standard Linear Regression.

3. Tree-based ensemble models

Random Forest (n_estimators=100, max_depth=10) - An ensemble of 100 independently trained decision trees. Each tree is trained on a bootstrap sample and uses a random subset of features at each split. The final prediction is the average across all trees. Max depth is limited to 10 to prevent overfitting.

Gradient Boosting - Builds trees sequentially, where each new tree corrects the residual errors of the previous ensemble. The learning rate controls how aggressively each tree contributes to the correction. These hyperparameters were obtained through prior tuning.

Configuration: n_estimators=204, learning_rate=0.151, max_depth=8, min_samples_split=4, min_samples_leaf=3.

4. Support Vector Regression

SVR (kernel='rbf', C=4, gamma=0.5, epsilon=0.1) - Support Vector Regression using a radial basis function (RBF) kernel. The cost parameter C=4 controls the trade-off between allowing training errors and model complexity. Gamma=0.5 defines how much influence a single training example has. SVR can capture nonlinear relationships but tends to be slower on large datasets.

5. Results

All five models were evaluated on the validation set. The code below trains each model and reports validation RMSE:

```
def evaluate_models(X_train, y_train, X_val, y_val):
    models = {
        "Gradient Boosting": GradientBoostingRegressor(
            n_estimators=204, learning_rate
=0.15091256567397734,
            max_depth=8, min_samples_split=4, min_samples_leaf
=3
        ),
        "Random Forest": RandomForestRegressor(
            n_estimators=100, max_depth=10, random_state=42
        ),
        "Ridge Regression": Ridge(alpha=1.0),
        "Linear Regression": LinearRegression(),
        "SVR": SVR(kernel='rbf', C=4, gamma=0.5, epsilon=0.1)
    }

    results = {}
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_val)
        rmse = mean_squared_error(y_val, y_pred) ** 0.5
        results[name] = {"model": model, "rmse": rmse, "y_pred"
: y_pred}
```

```
best_model_name = min(results, key=lambda x: results[x]["rmse"])
return best_model, best_model_name, results
```

The validation RMSE results for the initial five models:

Model	Validation RMSE
Gradient Boosting	39.5529
Random Forest	44.2852
Ridge Regression	52.4083
Linear Regression	52.4149
SVR	53.0525

Table 1: Validation RMSE for baseline and ensemble models

The best model among the initial five is **Gradient Boosting** with an RMSE of **39.5529**. Tree-based ensemble methods significantly outperformed the linear models, confirming that the relationship between building features and energy consumption is nonlinear and involves complex feature interactions.

C. Advanced Models - XGBoost and LightGBM

1. XGBoost

XGBoost (Extreme Gradient Boosting) is an optimized implementation of gradient boosting that includes regularization, parallel processing, and efficient handling of missing values.

Configuration:

- n_estimators: 200
- learning_rate: 0.1
- max_depth: 6
- subsample: 0.8 (row sampling to reduce overfitting)
- colsample_bytree: 0.8 (column sampling per tree)

```
def train_xgboost(X_train, y_train, X_val, y_val):
    xgb_model = xgb.XGBRegressor(
        n_estimators=200, learning_rate=0.1, max_depth=6,
        subsample=0.8, colsample_bytree=0.8, random_state=42
    )
    xgb_model.fit(X_train, y_train)
    y_pred = xgb_model.predict(X_val)
    rmse = mean_squared_error(y_val, y_pred) ** 0.5
    return xgb_model, rmse
```

2. LightGBM

LightGBM uses a histogram-based algorithm for splitting, which makes it faster than traditional gradient boosting on large datasets. It also uses leaf-wise tree growth (as opposed to level-wise), which can produce deeper, more accurate trees.

Configuration:

- `n_estimators`: 200
- `learning_rate`: 0.1
- `max_depth`: 6
- `num_leaves`: 31
- `subsample`: 0.8
- `colsample_bytree`: 0.8

```
def train_lightgbm(X_train, y_train, X_val, y_val):
    lgb_model = lgb.LGBMRegressor(
        n_estimators=200, learning_rate=0.1, max_depth=6,
        num_leaves=31, subsample=0.8, colsample_bytree=0.8,
        random_state=42
    )
    lgb_model.fit(X_train, y_train)
    y_pred = lgb_model.predict(X_val)
    rmse = mean_squared_error(y_val, y_pred) ** 0.5
    return lgb_model, rmse
```

3. Comparison with initial models

After training XGBoost and LightGBM, all seven models are compared. The final best model is selected by comparing validation RMSE across all models:

```
all_rmse = {name: results["rmse"] for name, results in
    all_results.items()}
best_model_name = min(all_rmse, key=all_rmse.get)
best_model = all_results[best_model_name]["model"]
```

The advanced gradient boosting models (XGBoost, LightGBM) are expected to compete closely with or outperform the scikit-learn Gradient Boosting implementation due to their optimizations and regularization capabilities.

D. Hyperparameter Tuning

1. Halving Grid Search for Gradient Boosting

To optimize the Gradient Boosting model, I use **HalvingGridSearchCV** from scikit-learn. This is a more efficient alternative to standard Grid Search: it starts by evaluating

all parameter combinations on a small subset of data, then progressively eliminates underperforming combinations while allocating more data to promising ones.

```
def tune_gradient_boosting(X_train, y_train):
    param_grid = {
        'n_estimators': [100, 200, 300],
        'learning_rate': [0.05, 0.1],
        'max_depth': [4, 6, 8],
    }
    halving_search = HalvingGridSearchCV(
        GradientBoostingRegressor(random_state=42),
        param_grid, cv=3,
        scoring='neg_root_mean_squared_error',
        verbose=1, n_jobs=-1, factor=2
    )
    halving_search.fit(X_train, y_train)
    return halving_search.best_estimator_
```

The tuning grid explores:

- **n_estimators** (100, 200, 300): the number of boosting stages. More trees can capture more complex patterns but risk overfitting.
- **learning_rate** (0.05, 0.1): controls the shrinkage applied to each tree's contribution. Lower rates require more trees but often generalize better.
- **max_depth** (4, 6, 8): the maximum depth of each individual tree. Deeper trees capture more complex interactions but are more prone to overfitting.

Cross-validation with 3 folds ensures that the selected parameters generalize well and are not overfit to a single train/validation split.

E. Visualization and Feature Analysis

1. RMSE comparison chart

A bar chart compares validation RMSE across all seven models, providing a clear visual ranking of model performance. This chart is saved as `model_comparison_rmse.png`.

```
plt.figure(figsize=(8, 5))
rmse_values = [results[name]['rmse'] for name in results]
sns.barplot(x=list(results.keys()), y=rmse_values)
plt.ylabel("Validation RMSE")
plt.title("Model Comparison - RMSE")
```

2. Predicted vs. actual plot

For the best model, a scatter plot of predicted vs. actual Site EUI values is generated. A 45-degree reference line (red dashed) represents perfect predictions. Points

clustered close to this line indicate strong model performance. This chart is saved as `predicted_vs_actual.png`.

```
plt.figure(figsize=(6, 6))
sns.scatterplot(x=y_val, y=best_pred)
plt.plot([y_val.min(), y_val.max()], [y_val.min(), y_val.max()], 'r--')
plt.xlabel("Actual site_eui")
plt.ylabel("Predicted site_eui")
```

3. Feature importance

For tree-based models, feature importance is extracted to identify the top 10 features driving energy consumption. The importance is based on how much each feature reduces the impurity (variance) across all trees in the ensemble. This chart is saved as `feature_importance.png`.

```
importances = best_model.feature_importances_
top_idx = importances.argsort()[-10:][::-1]
top_features = [X.columns[i] for i in top_idx]
sns.barplot(x=importances[top_idx], y=top_features, palette="viridis")
```

Expected top features include floor area, facility type, energy star rating, and climate variables such as heating/cooling degree days and temperature extremes.

4. Feature correlation with target

A separate bar chart shows the Pearson correlation of the top 10 most important features with the target variable. This helps distinguish between features that increase energy use (positive correlation) and those that decrease it (negative correlation). This chart is saved as `feature_correlations.png`.

5. Top predicted energy consumers

The final visualization ranks the top 10 buildings in the test set by predicted Site EUI. These represent the buildings a Sustainability Director would investigate first for energy audits or retrofits. This chart is saved as `top_predicted_buildings.png`.

F. Conclusion

1. Summary of model performance

Seven models were trained and compared for predicting Site Energy Usage Intensity:

Gradient boosting methods (Gradient Boosting, XGBoost, LightGBM) consistently outperformed linear models and SVR. This confirms that the relationship between build-

Model	Type	Key Characteristics	RMSE
Gradient Boosting	Boosting ensemble	204 trees, sequential error correction	39.5529
Random Forest	Bagging ensemble	100 trees, max depth 10	44.2852
Ridge Regression	Regularized linear	L2 penalty, handles multicollinearity	52.4083
Linear Regression	Linear baseline	Simple, interpretable, no regularization	52.4149
SVR	Kernel-based regression	RBF kernel, C=4, gamma=0.5	53.0525
XGBoost	Optimized boosting	Regularized, parallel, subsample 0.8	-
LightGBM	Histogram-based boosting	Leaf-wise growth, fast training	-

Table 2: Summary of all trained models (sorted by RMSE)

ing characteristics, climate variables, and energy consumption is **nonlinear** and involves **complex feature interactions** that tree-based ensemble methods capture well.

2. Key insights

- **Building characteristics matter most:** Features like floor area, facility type, and year built are among the strongest predictors of energy use.
- **Climate is a major driver:** Heating and cooling degree days, temperature extremes, and snowfall significantly influence energy consumption, confirming that geographic and climate context cannot be ignored.
- **Ensemble methods excel:** The performance gap between linear models and ensemble methods demonstrates the value of capturing nonlinear patterns and feature interactions.

3. Practical applications

The best model serves as a **portfolio-level screening tool** for Sustainability Directors:

- **Identify inefficient buildings:** Buildings with actual Site EUI significantly higher than predicted may have operational or structural inefficiencies.
- **Prioritize retrofits:** The top predicted energy consumers can be targeted for energy audits.
- **Separate climate from inefficiency:** By accounting for climate variables in the model, the predictions help distinguish between high energy use driven by extreme climates and high energy use driven by poor building performance.

4. Limitations and future work

- **No temporal features:** The model does not capture seasonal or monthly energy patterns, as the data is aggregated annually.
- **Label Encoding:** While sufficient for tree-based models, target encoding or one-hot encoding may improve performance for linear models.
- **Additional tuning:** Only Gradient Boosting was formally tuned. Applying hyperparameter optimization (e.g., Optuna, Bayesian search) to XGBoost and LightGBM

could yield further improvements.

- **Stacking/blending:** Combining predictions from multiple models through stacking could improve overall accuracy.

Appendix: How to Run

```
# Install dependencies
pip install -r requirements.txt

# Run the full pipeline
python building_energy_consumption_prediction.py
```

The script will:

1. Load and preprocess the training and test data from the `data/` directory
2. Train and evaluate all seven models
3. Generate visualization plots (saved as PNG files)
4. Output predictions to `submission_best_model.csv`

Dataset source: WiDS Datathon 2022 - <https://www.kaggle.com/competitions/widsdatathon2022/overview/citation>